

Sallen-Key Filters

Carlos Elias-Sura

I. PROJECT OVERVIEW

For this subsystem in the final project, I had to build both low-pass and high-pass Sallen-Key filters. The filters are designed to process signals below the low cut-off frequency and above the high cut-off frequency. The cut-off frequencies and other project requirements were designed around analyses of a certain song.

II. HIGH AND LOW PASS FILTERS

The high and low pass filters will detect low and high frequencies throughout the song "Chicago" by Michael Jackson. I set the corner frequencies to be 110 Hz and 800 Hz, which should be at least 200 Hz apart from each other to ensure that there was no overlap between the corners. The high pass filter will output signals above 800 Hz, while minimizing the output from an input with a frequency below this cut-off. The low pass filter does the opposite: it outputs signals below 110 Hz, while minimizing signals above the cut-off. The second order, underdamped circuit can be achieved with capacitors and resistors to achieve the desired quality factor of 0.707, and prevent peaking phenomenon from occurring, ensuring a smoother output.

A. Design and Analysis

In order to achieve a quality factor as close as possible to 0.707, I used a Jupyter Notebook file, as seen with the code in Appendix A, to go through each of the available resistor and capacitor values that will give me a cut off frequency within 5% of 110 for the low pass filter, and the same tolerance for the 800 Hz high pass filter. I also used a tolerance of 2.5% for the quality factor for both filters. In order to calculate the cut-off frequencies and the quality factors of each filter, the following equations from Texas Instrument used along the components seen in the Sallen-Key schematic in Figure 6 [1]:

$$f_{low} = \frac{1}{2\pi\sqrt{R_6 \cdot R_7 \cdot C_7 \cdot C_8}} \quad (1)$$

$$Q_{low} = \frac{\sqrt{R_6 \cdot R_7 \cdot C_7 \cdot C_8}}{R_6 \cdot C_7 + R_7 \cdot C_7} \quad (2)$$

$$f_{high} = \frac{1}{2\pi\sqrt{R_8 \cdot R_9 \cdot C_9 \cdot C_{10}}} \quad (3)$$

$$Q_{high} = \frac{\sqrt{R_8 \cdot R_9 \cdot C_9 \cdot C_{10}}}{R_9 \cdot C_{10} + R_9 \cdot C_9} \quad (4)$$

In the Texas Instrument document, 2 additional resistors are used, R_3 and R_4 , which are not present in my design, so if R_3 has infinite resistance, and R_4 has no resistance, the equation in the documents can be simplified to attain the equations above [1]. I used the program to generate a list of several components that result in my desired cut-off frequencies and quality factors within the acceptable range. Then, I realized that there is a trade-off between the quality factor and cut-off frequency, both can not be accurate, either one is closer to the true value or the other, but not both. So I proceeded to choose the following values, which had the most accurate frequencies and quality factors:

- 1) High Pass: $C_1 = 22$ nF, $C_2 = 1.5$ nF, $R_1 = 100$ k Ω , $R_2 = 12$ k Ω , yielding $f_{high} \approx 799.78$ Hz and $Q_{high} \approx 0.706$
- 2) Low Pass: $C_1 = 22$ nF, $C_2 = 47$ nF, $R_1 = 56$ k Ω , $R_2 = 33$ k Ω , yielding $f_{low} \approx 115.14$ Hz and $Q_{low} \approx 0.706$

Now, considering the transfer functions:

$$H(s)_{lo} = \frac{1}{s^2(R_6 R_7 C_7 C_8) + s(R_6 C_7 + R_7 C_7) + 1} \quad (5)$$

$$H(s)_{hi} = \frac{s^2(R_8 R_9 C_9 C_{10})}{s^2(R_8 R_9 C_9 C_{10}) + s(R_8 C_{10} + R_9 C_9) + 1} \quad (6)$$

Evaluating both at $H(s = j\omega)$, and plugging in $\omega = 2\pi \cdot 110$ for $H(j\omega)_{lo}$, and $\omega = 2\pi \cdot 800$ for $H(j\omega)_{hi}$, results in magnitudes of the transfer functions approximately equal to 0.706 and 0.707 respectively, which not only shows that they match the quality factors, but provides a datapoint to test in both the simulations and experimental analyses - the gain of -3 dB, which results from the transfer function at the cut off frequency.

B. Simulation and Experimental Results

After modeling both Sallen-Key filter circuits in KiCad, I ran a transient and AC simulation to see whether the time domain output worked, and if the bode plots looked as expected. To test the time domain output, I tested input signals at 100 Hz and 1000 Hz, all at an amplitude of 3 V. In addition to the time domain, the frequency domain plots were also generated and evaluated to determine where the cut-off frequencies are present, which were all in accordance with the analysis. Using a time step of 100 ns, an initial time of 1 ms, and final time of 20 ms for 100 Hz, and step of 50 ns, an initial time of 1 ms, and final time of 4 ms for 1 kHz, produced the expected results:

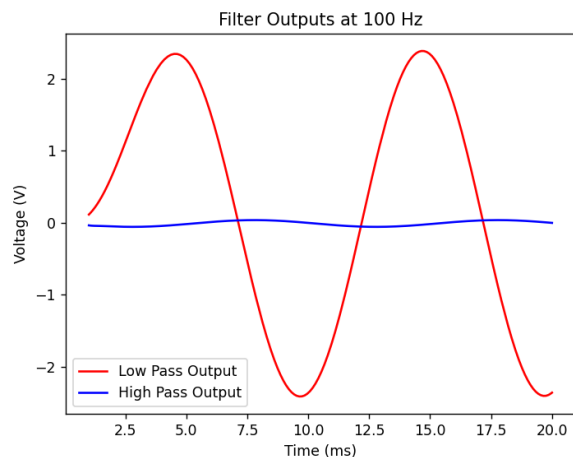


Fig. 1. Low and High Pass Filter outputs at 100 Hz.

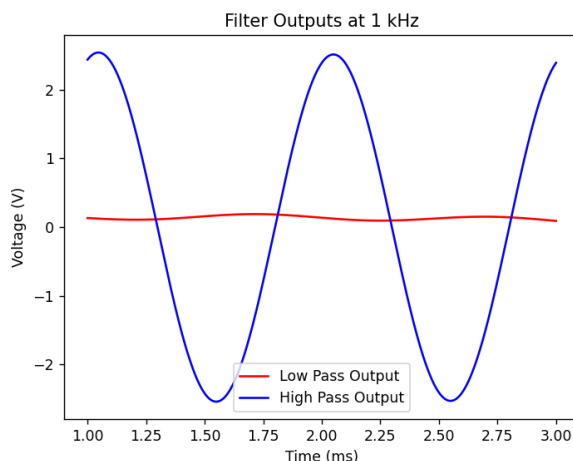


Fig. 2. Low and High Pass Filter outputs at 1 kHz.

Then, the AC analysis on KiCad, with 100 points per decade, starting at 10 Hz and finalizing at 25 kHz, showed the bode plots of each filter as shown below:

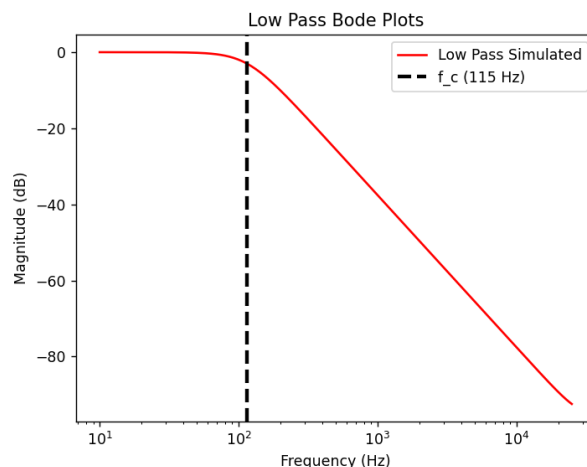


Fig. 3. Low Pass Filter Bode Plot from KiCad's AC simulation, at a gain of -3 dB, the frequency is 115 Hz, matching with the calculated value.

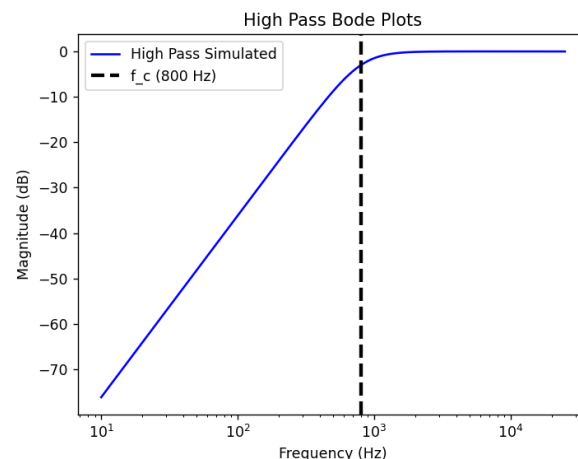


Fig. 4. High Pass Bode Plot from KiCad's AC simulation, at a gain of -3 dB, the frequency is approximately 800 Hz, also matching with the calculated value.

Once, the KiCad simulations provided satisfactory results, I built the circuit and hooked it up to the AD2. I first checked time-domain signals with the same input configurations as the simulation, which all returned similar outputs. Then, the bode plots were formed using Waveforms. Using a python script, as seen in Appendix B, I was able to plot both the simulated and experimental results onto the same graph, which were nearly overlapping. I checked the gain at -3 dB once again, and found the same cut-off frequencies as expected from both calculations and the simulations. There is some peaking observed in the experimental bode plot of the low pass filter, which can be attributed to not only the op-amp not working as an ideal op-amp, but also the tolerances of the capacitors or resistors which may affect performance at higher frequencies.

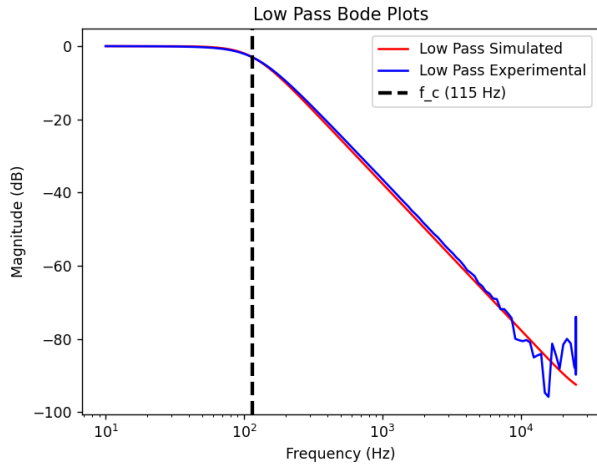


Fig. 5. Both simulated and experimental low pass filter bode plots on the same graph, both have a cut-off frequency of 115 Hz.

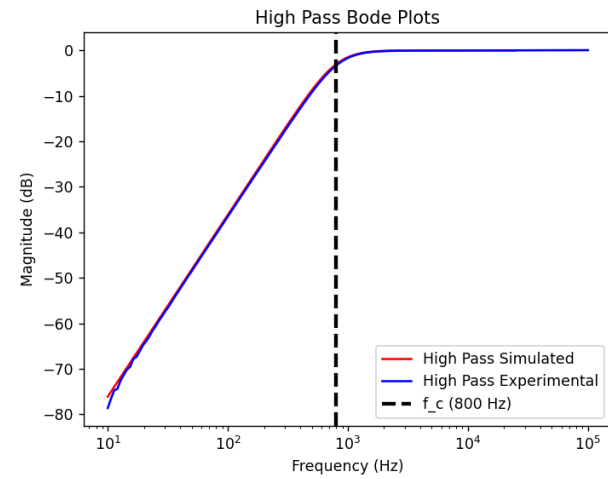
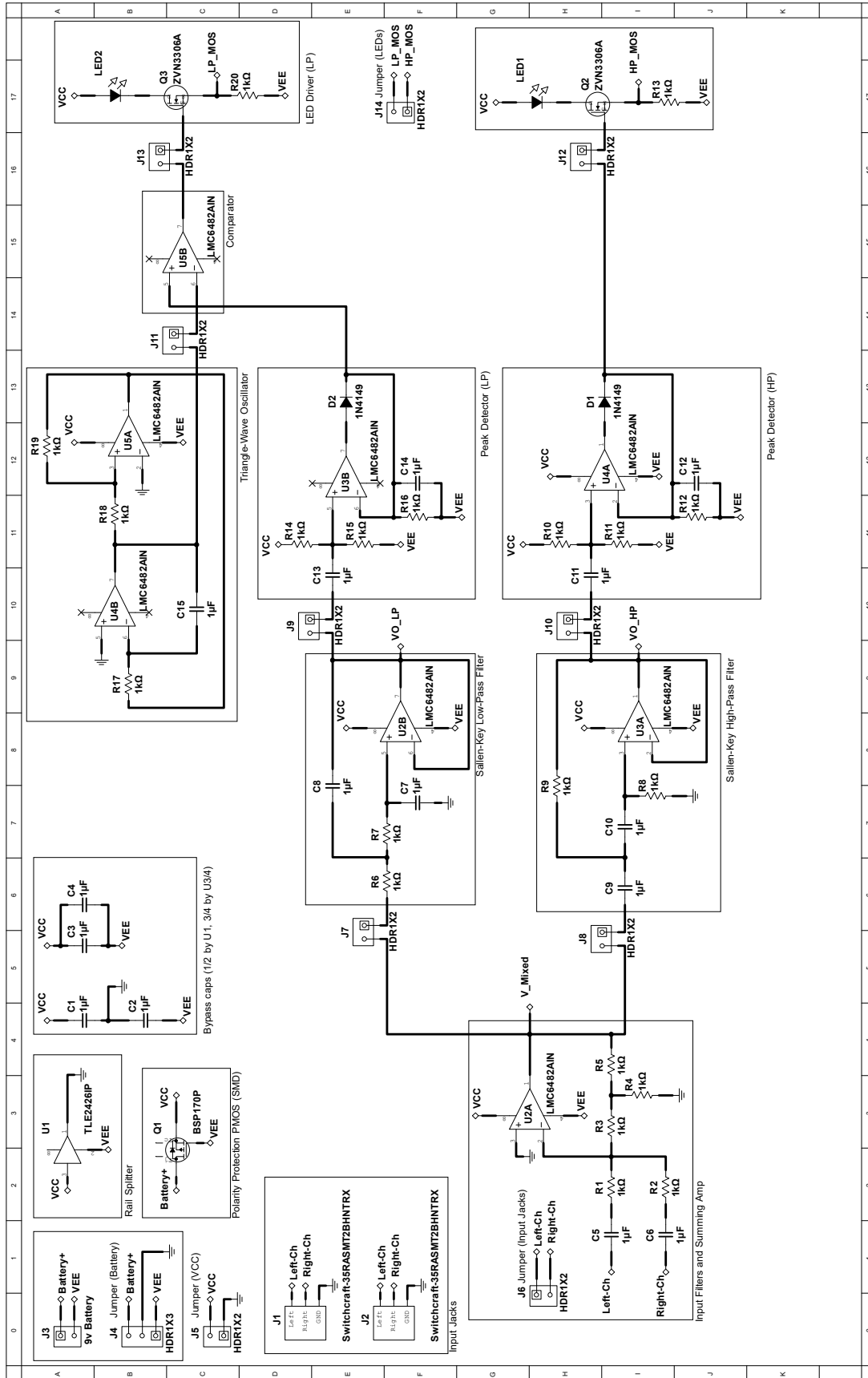


Fig. 6. Both simulated and experimental high pass filter bode plots on the same graph, both data have the same corner frequency of 800 Hz.

REFERENCES

- [1] *Analysis of the sallen-key architecture (sloa024b)*, Texas Instruments Incorporated, Sep. 2002. [Online]. Available: <https://www.ti.com/lit/an/sloa024b/sloa024b.pdf>.

Fig. 7. This figures shows all of the subsystems that are working together in analyzing the input signal and detect low and high frequencies of the song.



APPENDIX A SALLEN-KEY COMPONENT VALUES

```
% import numpy as np
% import sys
% import math
% import pandas as pd

% Rvals = [10000,12000,15000,18000,22000,27000,33000,39000,47000,56000,
68000,82000,100000,
%          120000,150000,180000,220000,270000,330000,390000,470000,560000,680000,
820000,1000000]
% CVals = [
%          10*10**-12,      #10pF
%          47*10**-12,      #47pF
%          0.1*10**-9,       #0.1nF
%          0.22*10**-9,      #0.22nF
%          0.33*10**-9,      #0.33nF
%          0.47*10**-9,      #0.47nF
%          0.68*10**-9,      #0.68nF
%          1*10**-9,         #1nF
%          1.5*10**-9,        #1.5nF
%          2.2*10**-9,        #2.2nF
%          3.3*10**-9,        #3.3nF
%          4.7*10**-9,        #4.7nF
%          6.8*10**-9,        #6.8nF
%          10*10**-9,         #10nF
%          15*10**-9,         #15nF
%          22*10**-9,         #22nF
%          33*10**-9,         #33nF
%          47*10**-9,         #47nF
%          68*10**-9,         #68nF
%          0.1*10**-6,        #0.1uF
%          0.15*10**-6,       #0.15uF
%          0.47*10**-6,       #0.47uF
%          1*10**-6,          #1uF
%          4.7*10**-6         #4.7uF
%          ]

% #This code selects values for high pass filter

% #These are the functions for calculating your high pass cutoff
frequency and Quality Factor
% #Use the functions in the Sallen-Kay Architecture pdf, and finish these functions
to calculate the cutoff frequency and quality factor given a set of component values
% def CutoffFrequency(C1, C2, R1, R2):
%     fc = 1 / (2 * math.pi * math.sqrt(R1 * R2 * C1 * C2)) # f0 equation from Sallen-Key
pdf, pg. 9
%     return(fc)

% def Quality_Factor(C1, C2, R1, R2):
%     Q = math.sqrt(R1 * R2 * C1 * C2) / (R2 * C2 + R2 * C1) # same here
%     return(Q)

% #To determine if a set of components creates an acceptable circuit, we will compare
```

```

the frequency and quality we calculate with the values we want
% #TO do this, we'll set an upper and lower bound that values we deem 'acceptable'
fall between

% freqdes = 800 #TODO make this your desired high pass cutoff frequency
% #This tolerance is used to set the high and low bound. For example, 0.1*freqdes
will save values within 10% of your desired frequency
% ftolerance = 0.05 * freqdes # 5% tolerance of desired frequency
% fmax = freqdes + ftolerance
% fmin = freqdes - ftolerance

% Qdes = 0.707 #TODO make this your desired quality factor
% #This tolerance is used to set the high and low bound. For example, 0.1*Qdes
will save values within 10% of your desired cutoff
% Qtolerance = 0.025 * Qdes # 2.5% tolerance
% Qmax = Qdes + Qtolerance
% Qmin = Qdes - Qtolerance

% #Creates and array that all the valid solutions will be saved to and printed
% solutions = []

% #Defines each component to be an array of every possible value
% R1 = Rvals
% R2 = Rvals
% C1 = Cvals
% C2 = Cvals

% #This loops through an array for every value, and checks the corner frequency and quality
factor for every possible combination of component values
% #If a set of values produces a corner frequency and quality within your chosen tolerance,
it will print out that set of values
% for r in range(len(R1)):
%     for R in range(len(R2)):
%         for c in range(len(C1)):
%             for C in range(len(C2)):
%                 #Calculates the corner and quality for the current set of values
%                 corner = CutoffFrequency(C1[c], C2[C], R1[r], R2[R])
%                 quality = Quality_Factor(C1[c], C2[C], R1[r], R2[R])

%                 #If the corner and cutoff that were calculated are within your bounds,
they're appended to the solution array
%                 if corner <= fmax and corner >= fmin and quality <= Qmax and quality >= Qmin:
%                     solutions.append(np.array([C1[c], C2[C], R1[r], R2[R], corner, quality]))

% #Puts the soluions into a dataframe to read
% df = pd.DataFrame(solutions, columns=['C1', 'C2', 'R1', 'R2', 'Corner f',
'Quality Factor'])
% print(df)

% #This code selects values for low pass filter

% #These are the functions for calculating your high pass cutoff
frequency and Quality Factor
% #Use the functions in the Sallen-Kay Architecture pdf, and finish these functions

```

```

to calculate the cutoff frequency and quality factor given a set of component values
% def CutoffFrequency(C1, C2, R1, R2):
%     fc = 1 / (2 * math.pi * math.sqrt(R1 * R2 * C1 * C2)) # pg. 4
%     return(fc)

% def Quality_Factor(C1, C2, R1, R2):
%     Q = math.sqrt(R1 * R2 * C1 * C2) / (R1 * C1 + R2 * C1 ) # ditto
%     return(Q)

% #To determine if a set of components creates an acceptable circuit, we will
compare the frequency and quality we calculate with the values we want
% #TO do this, we'll set an upper and lower bound that values we deem 'acceptable'
fall between

% freqdes = 110 #TODO make this your desired low pass cutoff frequency
% #This tolerance is used to set the high and low bound. For example, 0.1*freqdes
will save values within 10% of your desired frequency
% ftolerance = 0.05 * freqdes # 5% tolerance
% fmax = freqdes + ftolerance
% fmin = freqdes - ftolerance

% Qdes = 0.707 #TODO make this your desired quality factor
% #This tolerance is used to set the high and low bound. For example, 0.1*Qdes
will save values within 10% of your desired cutoff
% Qtolerance = 0.025 * Qdes # 2.5% tolerance
% Qmax = Qdes + Qtolerance
% Qmin = Qdes - Qtolerance

% #Creates an array that all the valid solutions will be saved to and printed
% solutions = []

% #Defines each component to be an array of every possible value
% R1 = Rvals
% R2 = Rvals
% C1 = Cvals
% C2 = Cvals

% #This loops through an array for every value, and checks the corner frequency and
quality factor for every possible combination of component values
% #If a set of values produces a corner frequency and quality within your chosen tolerance,
it will print out that set of values
% for r in range(len(R1)):
%     for R in range(len(R2)):
%         for c in range(len(C1)):
%             for C in range(len(C2)):
%                 #Calculates the corner and quality for the current set of values
%                 corner = CutoffFrequency(C1[c], C2[C], R1[r], R2[R])
%                 quality = Quality_Factor(C1[c], C2[C], R1[r], R2[R])

%                 #If the corner and cutoff that were calculated are within your bounds,
they're appended to the solution array
%                 if corner <= fmax and corner >= fmin and quality <= Qmax and quality >= Qmin:
%                     solutions.append(np.array([C1[c], C2[C], R1[r], R2[R], corner, quality]))

```

```
% #Puts the soluions into a dataframe to print and read the values you found
% df = pd.DataFrame(solutions, columns=['C1', 'C2', 'R1', 'R2', 'Corner f',
'Quality Factor'])
% print(df)
```

APPENDIX B PYTHON PLOT SCRIPT

```
## plots simulated and experimental results of Bode Plots for low-
and high-pass Sallen-Key filters
% # By Carlos Elias-Sura
% # 3/13/26

% import pandas as pd
% import matplotlib.pyplot as plt
% # read simulated and experimental data
% highPass_experimental = pd.read_csv("Bode Plot Data/High Pass
Experimental Bode.csv")
% lowPass_experimental = pd.read_csv("Bode Plot Data/Low Pass
Experimental Bode.csv")

% simulatedData = pd.read_csv("Bode Plot Data/Sallen Key
Filters Simulation Bode.csv", sep = ';') # includes both low
and high pass filter data

% # plot both experimental and simulated results into 2 graphs, one for
high pass, another for low pass
% # Referenced: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.bode.html
% # to plot frequency and magnitude data using a log scale
% plt.figure()

% # Referenced: https://matplotlib.org/2.0.2/api/pyplot\_api.html#matplotlib.pyplot.semilogx
% # to use appropriate args like color, title, etc.
% plt.semilogx(simulatedData["frequency"],
%             simulatedData["V(/Vout1) (gain)"],
%             color = 'red',
%             label = 'Low Pass Simulated')
% plt.semilogx(lowPass_experimental["Frequency (Hz)"],
%             lowPass_experimental["Channel 1 Magnitude (dB)"],
%             color = 'blue',
%             label = 'Low Pass Experimental')

% plt.xlabel("Frequency (Hz)")
% plt.ylabel("Magnitude (dB)")
% plt.legend()

% plt.title("Low Pass Bode Plots")
% plt.show()

% # high pass simulated and experimental
% plt.figure()
% plt.semilogx(simulatedData["frequency"],
%             simulatedData["V(/Vout2) (gain)"],
```



```

%             color = "red",
%             label = "High Pass Simulated")
% plt.semilogx(highPass_experimental["Frequency (Hz)"],
%             highPass_experimental["Channel 1 Magnitude (dB)"],
%             color = "blue",
%             label = "High Pass Experimental")

% plt.xlabel("Frequency (Hz)")
% plt.ylabel("Magnitude (dB)")
% plt.legend()

% plt.title("High Pass Bode Plots")
% plt.show()

```